CSC 251 Lecture 15 – More Modeling and Combining Data

Simple Linear Regression Review

Last time, we used the `statsmodels` package to obtain a summary of our simple (OLS) regression results. One predictor (x) variable was regressed against a continuous response variable (y).

```
results1 = smf.ols(formula = 'tip ~ total_bill', data = tips).fit()
```

We also:

- Created a graphical representation using `matplotlib` and `Seaborn`:
- Found the correlation coefficient, r and R^2
- Wrote the equation of the regression line

Other Regression Models

Types of models include:

- simple – one variable as predictor
- multiple – several variables as predictors
- logistic – for dichotomous categorical variables
- robust – reduces the effects of outliers
- polynomial – fits to a higher order curve
- lowess – locally weighted scatterplot smoothing technique

Multiple Regression

Example 1: Multiple predictors can be regressed with the same response variable, where each parameter is interpreted with "all other variables held constant." Fits to a linear equation:

$$y = m_1 * x_1 + m_2 * x_2 + b$$

```
results2 = smf.ols(formula = 'tip ~ total_bill + size', data = tips).fit()
```

OLS Regression Results

Dep. Variable:	tip	R-squared:	0.468
Model:	OLS	Adj. R-squared:	0.463
Method:	Least Squares	F-statistic:	105.9
Date:	Mon, 06 Nov 2023	Prob (F-statistic):	9.67e-34
Time:	15:55:29	Log-Likelihood:	-347.99
No. Observations:	244	AIC:	702.0
Df Residuals:	241	BIC:	712.5
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.6689	0.194	3.455	0.001	0.288	1.050
total_bill	0.0927	0.009	10.172	0.000	0.075	0.111
size	0.1926	0.085	2.258	0.025	0.025	0.361

Omnibus:	24.753	Durbin-Watson:	2.100
Prob(Omnibus):	0.000	Jarque-Bera (JB):	46.169
Skew:	0.545	Prob(JB):	9.43e-11
Kurtosis:	4.831	Cond. No.	67.6

Our predictive equation:

tip = 0.0927 * total_bill + 0.1926 * size + 0.6689

so for a party of 10 with a total bill of $100, the tip would be:

tip = 0.0927 * (100) + 0.1926 * 10 + 0.6689 = 11.8649

which translates to a tip of $11.86.

To obtain our parameters, conveniently packaged into a Series:

```
results2.params
```

```
Intercept     0.668945
total_bill    0.092713
size          0.192598
dtype: float64
```

To graph, add a column based upon the multiple regression:

```
tips['mult'] = tips['total_bill'] * results2.params.iloc[1] + tips['size'] * results2.params.iloc[2]
      + results2.params.iloc[0]
```

What does the graph look like?

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
```

Add data points to the 3d scatter plot:

```
ax.scatter(tips["total_bill"], tips["size"], tips["mult"])
```

Create a meshgrid for the plane:

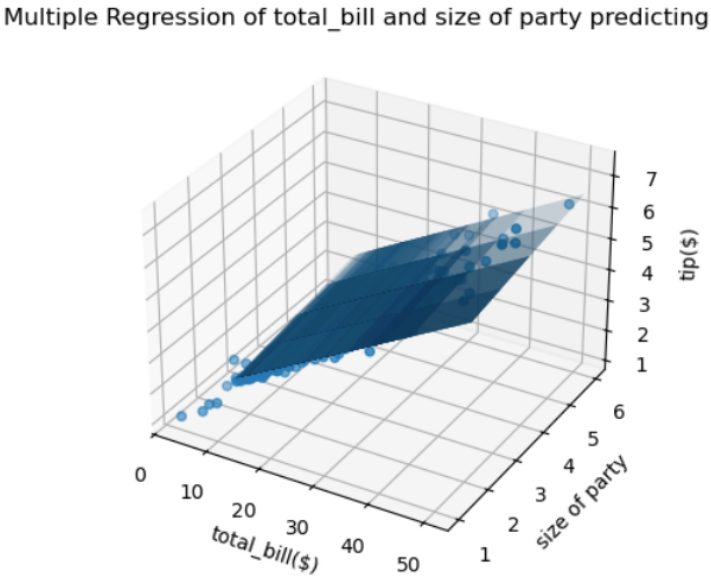
```
x_plane, y_plane = np.meshgrid(tips["total_bill"], tips["size"])
z_plane = results2.params[1] * x_plane + results2.params[2] * y_plane + results2.params[0]
```

Plot the plane with selected transparency value:

```
ax.plot_surface(x_plane, y_plane, z_plane, alpha=0.5)
```

Title and label graph:

```
ax.set_title('Multiple Regression of total_bill and size of party predicting tip')
ax.set_xlabel('total_bill($)')
ax.set_ylabel('size of party')
ax.set_zlabel('tip($)')
```



Multiple, Multiple Regression

Example 2: Let's add more variables to our multiple regression:

```
results3 = smf.ols(formula = 'tip ~ total_bill + size + smoker + day + time', data = tips).fit()
```

Does it improve the correlation?

OLS Regression Results

Dep. Variable:	tip	R-squared:	0.470
Model:	OLS	Adj. R-squared:	0.454
Method:	Least Squares	F-statistic:	29.89
Date:	Mon, 06 Nov 2023	Prob (F-statistic):	2.15e-29
Time:	15:55:50	Log-Likelihood:	-347.50
No. Observations:	244	AIC:	711.0
Df Residuals:	236	BIC:	739.0
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.7896	0.346	2.279	0.024	0.107	1.472
smoker[T.Yes]	-0.0865	0.146	-0.591	0.555	-0.375	0.202
day[T.Sat]	-0.1259	0.309	-0.408	0.684	-0.734	0.482
day[T.Sun]	-0.0326	0.319	-0.102	0.919	-0.661	0.596
day[T.Thur]	-0.1609	0.393	-0.410	0.682	-0.934	0.612
time[T.Lunch]	0.0681	0.444	0.154	0.878	-0.806	0.942
total_bill	0.0943	0.010	9.884	0.000	0.075	0.113
size	0.1764	0.089	1.974	0.050	0.000	0.352

Omnibus:	27.521	Durbin-Watson:	2.099
Prob(Omnibus):	0.000	Jarque-Bera (JB):	51.508
Skew:	0.603	Prob(JB):	6.53e-12
Kurtosis:	4.901	Cond. No.	205.

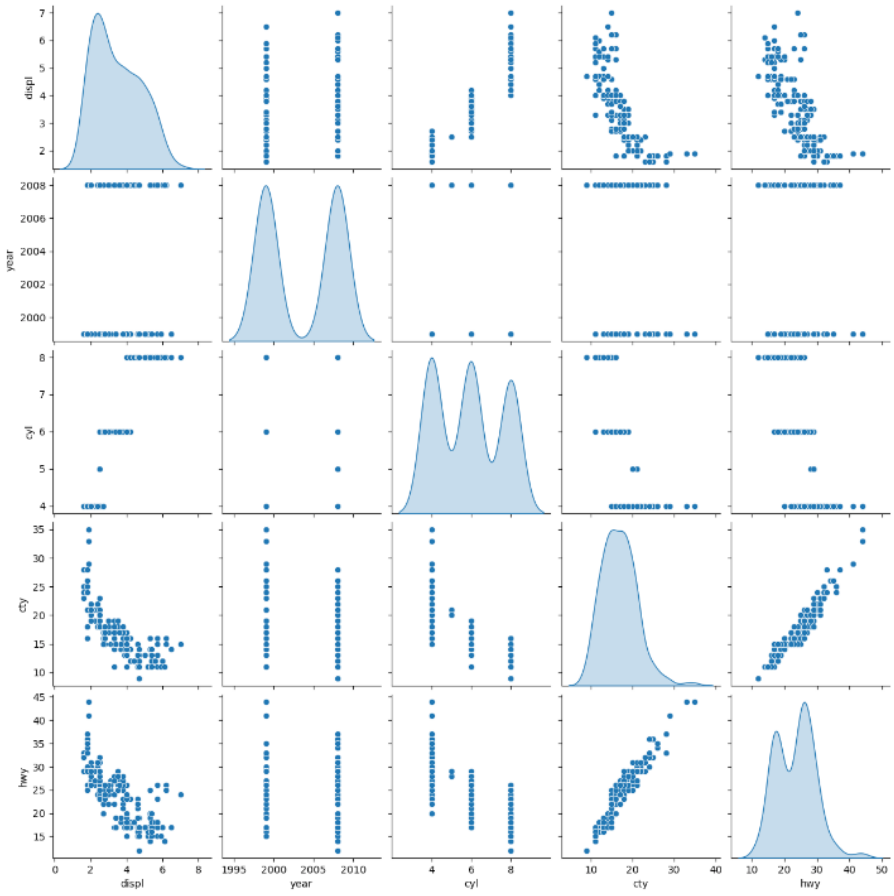
What's happening with the categorical variables?

Dummy variable for binary features are auto-created with `statsmodels.` One value is dropped as a reference.

What's the equation?

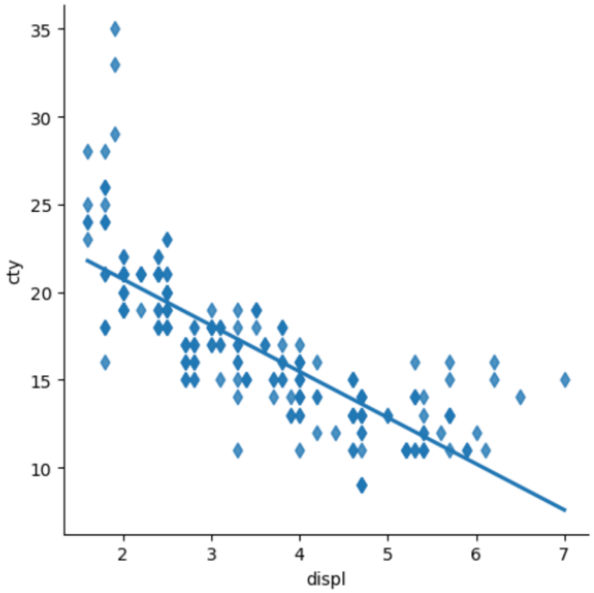
Polynomial Fit

Recall the pairplot of the mpg auto data:



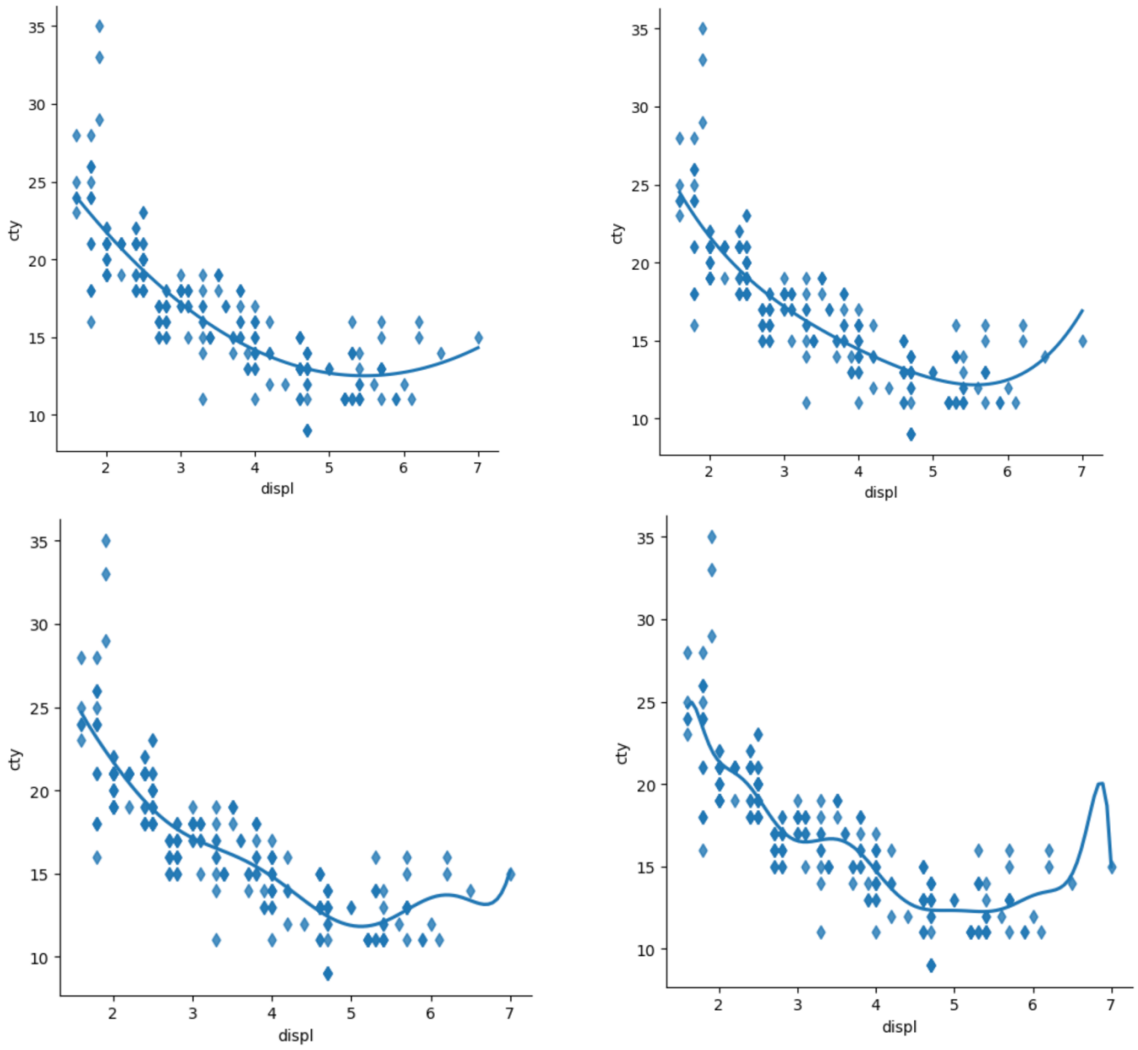
Example 3a: Let's look at the relationship between engine displacement (displ) and city gas mileage (cty). Is a linear model a best fit?

```
import seaborn as sns
sns.lmplot(data=mpg, x='displ', y = 'cty', ci=None, markers='d')
```



Consider a polynomial fit, first order 2, then 4, 8, then 12. Is higher order better?

```
sns.lmplot(data=mpg, x='displ', y = 'cty', ci=None, markers='d', order= 2)
```



Underfitting versus overfitting

Example 3b: To access the equation, first fit the curve using the NumPy `polyfit` method, indicating degree 2:

```
coefficients = np.polyfit(mpg['displ'], mpg['cty'], 2)
[ 0.76826417, -8.40223672, 35.49277815]
```

Convert the raw coefficients into a callable polynomial function object:

```
polynomial = np.poly1d(coefficients)
print(polynomial)

         2
0.7683 x - 8.402 x + 35.49
```

Create a new column with the predicted values:

```
mpg['fit2']=polynomial(mpg['displ'])
mpg
```

Plot on same axes. (see notebook)

Combining Data Sets

Often your data exists in different files and must be combined into the same "table" or DataFrame for analysis and visualization. Data can be combined by:

1) Concatenation – appending a row(s) or column(s) using pandas

Example 4: Suppose we imported two DataFrames:

	A	B	C
0	a0	b0	c0
1	a1	b1	c1
2	a2	b2	c2
3	a3	b3	c3

	A	B	C
0	a4	b4	c4
1	a5	b5	c5
2	a6	b6	c6
3	a7	b7	c7

```
row_concat = pd.concat([df1, df2])
```

	A	B	C
0	a0	b0	c0
1	a1	b1	c1
2	a2	b2	c2
3	a3	b3	c3
0	a4	b4	c4
1	a5	b5	c5
2	a6	b6	c6
3	a7	b7	c7

To reset the row index:

```
row_concat = pd.concat([df1, df2], ignore_index=True)
```

	A	B	C
0	a0	b0	c0
1	a1	b1	c1
2	a2	b2	c2
3	a3	b3	c3
4	a4	b4	c4
5	a5	b5	c5
6	a6	b6	c6
7	a7	b7	c7

By default, will add rows since axis = 0. To concatenate a df to columns:

```
col_concat = pd.concat([df1, df2], axis=1)
```

	A	B	C	A	B	C
0	a0	b0	c0	a4	b4	c4
1	a1	b1	c1	a5	b5	c5
2	a2	b2	c2	a6	b6	c6
3	a3	b3	c3	a7	b7	c7

As before, to reset the index:

```
col_concat = pd.concat([df1, df2], axis=1, ignore_index=True)
```

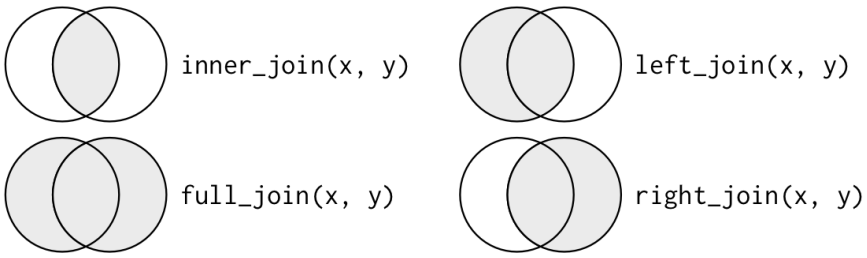
	0	1	2	3	4	5
0	a0	b0	c0	a4	b4	c4
1	a1	b1	c1	a5	b5	c5
2	a2	b2	c2	a6	b6	c6
3	a3	b3	c3	a7	b7	c7

We know how to add a new column using a vector in NumPy:

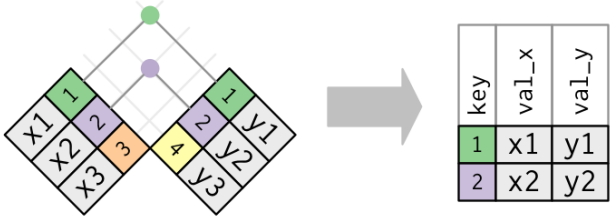
```
col_concat['newcol'] = ['a', 'b', 'c', 'd']
```

	A	B	C	A	B	C	newcol
0	a0	b0	c0	a4	b4	c4	a
1	a1	b1	c1	a5	b5	c5	b
2	a2	b2	c2	a6	b6	c6	c
3	a3	b3	c3	a7	b7	c7	d

2) Merging



An inner join matches pairs of observations with equal keys:



Example 5: Suppose we want to merge the following DataFrames:

`pets`

	petname	months	weight
0	Spot	4	2.1
1	Spot	8	5.8
2	Spot	10	7.5
3	Spot	14	8.9
4	Spot	18	9.2
5	Lassie	4	6.0
6	Lassie	8	15.1
7	Lassie	12	21.9
8	Lassie	14	23.0
9	Lassie	16	24.1
10	Fido	3	10.2
11	Fido	8	64.1
12	Fido	10	89.3
13	Fido	14	99.3
14	Fido	18	102.1

`breeders`

	petname	breeder
0	Spot	JK Kennels
1	Clifford	Public Breeders
2	Lassie	CBS Studios
3	Monica	Home Free

`owners`

	Dogname	ownername
0	Spot	Jenn Smith
1	Clifford	Red Big
2	Lassie	Rudd Weatherwax
3	Monica	FreeMom Sally

Let's try:

```
petbreeder_merge = pets.merge(breeders)
```

	petname	months	weight	breeder
0	Spot	4	2.1	JK Kennels
1	Spot	8	5.8	JK Kennels
2	Spot	10	7.5	JK Kennels
3	Spot	14	8.9	JK Kennels
4	Spot	18	9.2	JK Kennels
5	Lassie	4	6.0	CBS Studios
6	Lassie	8	15.1	CBS Studios
7	Lassie	12	21.9	CBS Studios
8	Lassie	14	23.0	CBS Studios
9	Lassie	16	24.1	CBS Studios

What's included? What's not?

Let's try:

```
pet_merge = pets.merge(owners)
```

Result?

```
MergeError: No common columns to perform merge on.
```

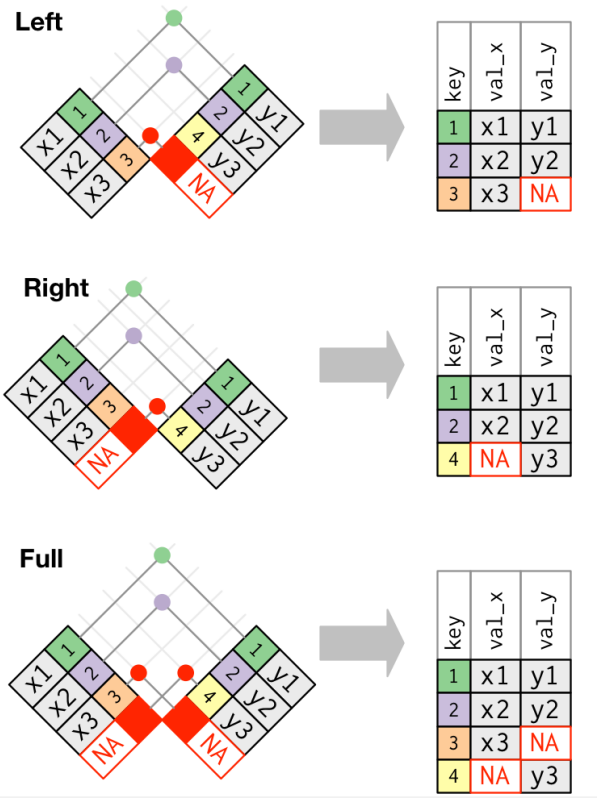
Solution?

```
pet_merge = pets.merge(owners, left_on='petname', right_on = 'Dogname')
```

	petname	months	weight	Dogname	ownername
0	Spot	4	2.1	Spot	Jenn Smith
1	Spot	8	5.8	Spot	Jenn Smith
2	Spot	10	7.5	Spot	Jenn Smith
3	Spot	14	8.9	Spot	Jenn Smith
4	Spot	18	9.2	Spot	Jenn Smith
5	Lassie	4	6.0	Lassie	Rudd Weatherwax
6	Lassie	8	15.1	Lassie	Rudd Weatherwax
7	Lassie	12	21.9	Lassie	Rudd Weatherwax
8	Lassie	14	23.0	Lassie	Rudd Weatherwax
9	Lassie	16	24.1	Lassie	Rudd Weatherwax

The merge will adjust to a one-to-one, many-to-one or many-to-many merge depending on duplicates and left/right indicators.

An outer join keeps observations that appear in at least one table:



https://r4ds.had.co.nz/relational-data.html

Example 6:

```
pet_merge_left = pets.merge(breeders, how = 'left')
```

	petname	months	weight	breeder
0	Spot	4	2.1	JK Kennels
1	Spot	8	5.8	JK Kennels
2	Spot	10	7.5	JK Kennels
3	Spot	14	8.9	JK Kennels
4	Spot	18	9.2	JK Kennels
5	Lassie	4	6.0	CBS Studios
6	Lassie	8	15.1	CBS Studios
7	Lassie	12	21.9	CBS Studios
8	Lassie	14	23.0	CBS Studios
9	Lassie	16	24.1	CBS Studios
10	Fido	3	10.2	NaN
11	Fido	8	64.1	NaN
12	Fido	10	89.3	NaN
13	Fido	14	99.3	NaN
14	Fido	18	102.1	NaN

```
pet_merge_outer = pets.merge(breeders, how = 'outer')
```

	petname	months	weight	breeder
0	Spot	4.0	2.1	JK Kennels
1	Spot	8.0	5.8	JK Kennels
2	Spot	10.0	7.5	JK Kennels
3	Spot	14.0	8.9	JK Kennels
4	Spot	18.0	9.2	JK Kennels
5	Lassie	4.0	6.0	CBS Studios
6	Lassie	8.0	15.1	CBS Studios
7	Lassie	12.0	21.9	CBS Studios
8	Lassie	14.0	23.0	CBS Studios
9	Lassie	16.0	24.1	CBS Studios
10	Fido	3.0	10.2	NaN
11	Fido	8.0	64.1	NaN
12	Fido	10.0	89.3	NaN
13	Fido	14.0	99.3	NaN
14	Fido	18.0	102.1	NaN
15	Clifford	NaN	NaN	Public Breeders
16	Monica	NaN	NaN	Home Free

Which is preferred?